Assignment-08

2303A51680

K Rishitha

Batch-23

Task - 1

(Username Validator – Apply AI in
Authentication Context)
• Task: Use AI to generate at least 3 assert test cases for a function is_valid_username(username) and then implement the function using Test-Driven Development principles.
• Requirements:
o Username length must be between 5 and 15 characters.
o Must contain only alphabets and digits.
o Must not start with a digit.
o No spaces allowed.
Example Assert Test Cases:
assert is_valid_username("User123") == True
assert is_valid_username("12User") == False
assert is_valid_username("Us er") == False
Expected Output #1:
• Username validation logic successfully passing all AI-generated test cases.

```python
def is_valid_username(username):
    if len(username) < 5 or len(username) > 15:
        return False
    if not username[0].isalpha():
        return False
    for char in username:
        if not(char.isalnum() or char =='_'):
            return False
    return True
#test cases for the is_valid_username function
assert is_valid_username("User123") == True
assert is_valid_username("12USer") == False
assert is_valid_username("Us er") == False
print("All test cases for is_valid_username passed!")
```

```
PS C:\Users\HP\Desktop\AI-Lab> & C:/Users/HP/AppData/Local/Programs/Python/Python313/python.exe c:/Users/HP/Desktop/AI-Lab/demo9.py
All test cases for is_valid_username passed!
PS C:\Users\HP\Desktop\AI-Lab>
```

Task - 2

(Even–Odd & Type Classification – Apply
AI for Robust Input Handling)

• Task: Use AI to generate at least 3 assert test cases for a function classify_value(x) and implement it using conditional logic and loops.

• Requirements:

o If input is an integer, classify as "Even" or "Odd".

o If input is 0, return "Zero".

o If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

assert classify_value(8) == "Even"

assert classify_value(7) == "Odd"

assert classify_value("abc") == "Invalid Input"

Expected Output #2:

• Function correctly classifying values and passing all test cases.

```python
def classify_value(x):
    if x < 0:
        return "Negative"
    elif x == 0:
        return "Zero"
    elif x%2 == 0:
        return "Even"
    else:
        return "Odd"
#test cases for the classify_value function
assert classify_value(8)=="Even"
assert classify_value(7)=="Odd"
assert classify_value("abce")=="Invalid Input"
```

```
Traceback (most recent call last):
  File "c:\Users\HP\Desktop\AI-Lab\demo9.py", line 91, in <module>
    assert classify_value("abce")=="Invalid Input"
           ~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^
  File "c:\Users\HP\Desktop\AI-Lab\demo9.py", line 80, in classify_value
    if x < 0:
       ^^^^^
TypeError: '<' not supported between instances of 'str' and 'int'
PS C:\Users\HP\Desktop\AI-Lab>
```

Task - 3

(Palindrome Checker – Apply AI for String Normalization)

• Task: Use AI to generate at least 3 assert test cases for a function is_palindrome(text) and implement the function.

• Requirements:

o Ignore case, spaces, and punctuation.

o Handle edge cases such as empty strings and single characters.

Example Assert Test Cases:

assert is_palindrome("Madam") == True

assert is_palindrome("A man a plan a canal Panama") ==True

assert is_palindrome("Python") == False

Expected Output #3:

• Function correctly identifying palindromes and passing all AI-generated tests.

```python
def is_palindrome(text):
    cleaned_text = ''.join(char.lower() for char in text if char.isalnum())
    return cleaned_text == cleaned_text[::-1]
#test cases for the is_palindrome function
assert is_palindrome("Madam")==True
assert is_palindrome("A man a plan a canal panama")==True
assert is_palindrome("Python")==False
print("All test cases for is_palindrome passed!")
```

```
PS C:\Users\HP\Desktop\AI-Lab> & C:/Users/HP/AppData/Local/Programs/Python/Python313/python.exe c:/Users/HP/Desktop/AI-Lab/demo9.py
All test cases for is_palindrome passed!
PS C:\Users\HP\Desktop\AI-Lab>
```

Task - 4

(Email ID Validation – Apply AI for Data Validation)

• Task: Use AI to generate at least 3 assert test cases for a function validate_email(email) and implement the function.

• Requirements:

o Must contain @ and .

o Must not start or end with special characters.

o Should handle invalid formats gracefully.

Example Assert Test Cases:

assert validate_email("user@example.com") == True

assert validate_email("userexample.com") == False

assert validate_email("@gmail.com") == False

Expected Output #5:

• Email validation function passing all AI-generated test cases and handling edge cases correctly.

```python
def validate_email(email):
    if "@" not in email or "." not in email:
        return False
    at_index=email.index('@')
    dot_index=email.rindex('.')
    if at_index < 1 or dot_index < at_index + 2 or dot_index >= len(email) - 1:
        return False
    return True
#test cases for the validate_email function
assert validate_email("user@example.com")==True
assert validate_email("userexample.com")==False
assert validate_email("@gmail.com")==False
assert validate_email("user@com")==False
print("All test cases for validate_email passed!")
```

```
PS C:\Users\HP\Desktop\AI-Lab> & C:/Users/HP/AppData/Local/Programs/Python/Python313/python.exe c:/Users/HP/Desktop/AI-Lab/demo9.py
All test cases for validate_email passed!
PS C:\Users\HP\Desktop\AI-Lab>
```

Task - 5

(Perfect Number Checker – Test Case Design)

• Function: Check if a number is a perfect number (sum of divisors = number).

• Test Cases to Design:

o Normal case: 6 → True, 10 → False.

o Edge case: 1.

o Negative number case.

o Larger case: 28.

• Requirement: Validate correctness with assertions.

```python
def is_perfect_number(num):
    if num < 1:
        return False
    divisors_sum=sum(i for i in range(1,num) if num % i == 0)
    return divisors_sum == num
#test cases for the is_perfect_number function
assert is_perfect_number(6)==True
assert is_perfect_number(10)==False
assert is_perfect_number(28)==True
assert is_perfect_number(1)==False
print("All test cases for is_perfect_number passed!")
```

```
PS C:\Users\HP\Desktop\AI-Lab> & C:/Users/HP/AppData/Local/Programs/Python/Python313/pyt
All test cases for is_perfect_number passed!
PS C:\Users\HP\Desktop\AI-Lab>
```

Task - 6

(Abundant Number Checker – Test Case Design)

• Function: Check if a number is abundant (sum of divisors > number).

• Test Cases to Design:

o Normal case: 12 → True, 15 → False.

o Edge case: 1.

o Negative number case.

o Large case: 945.

Requirement: Validate correctness with unittest

```python
def Abundant_Number_Checker(num):
    if num < 1:
        return False
    divisors_sum=sum(i for i in range(1,num) if num % i == 0)
    return divisors_sum > num
import unittest
class TestAbundantNumberChecker(unittest.TestCase):
    def test_abundant(self):
        self.assertTrue(Abundant_Number_Checker(12))
        self.assertTrue(Abundant_Number_Checker(15))
        self.assertTrue(Abundant_Number_Checker(1))
        self.assertFalse(Abundant_Number_Checker(-1))
        self.assertTrue(Abundant_Number_Checker(945))
if __name__ == '__main__':
    unittest.main()
```

```
, line 146, in test_abundant
    self.assertTrue(Abundant_Number_Checker(15))
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
AssertionError: False is not true

----------------------------------------------------------------------
Ran 1 test in 0.002s

FAILED (failures=1)
```

Task - 7

(Deficient Number Checker – Test Case Design)

• Function: Check if a number is deficient (sum of divisors < number).

• Test Cases to Design:

o Normal case: 8 → True, 12 → False.

o Edge case: 1.

o Negative number case.

o Large case: 546.

Requirement: Validate correctness with pytest.

```python
def deficient_number_checker(num):
    if num < 1:
        return False
    divisors_sum=sum(i for i in range(1,num) if num % i == 0)
    return divisors_sum < num
def test_deficient_number_checker():
    assert deficient_number_checker(8)==True
    assert deficient_number_checker(12)==False
    assert deficient_number_checker(1)==True
    assert deficient_number_checker(546)==False
    print("All test cases for deficient_number_checker passed!")
```

```
PS C:\Users\HP\Desktop\AI-Lab> & C:/Users/HP/AppData/Local/Programs/Python/Python313/python.exe c:/Users/HP/Desktop/AI-Lab/demo9.py
PS C:\Users\HP\Desktop\AI-Lab> python -m pytest demo9.py
========================================================== test session starts ==========================================================
platform win32 -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\HP\Desktop\AI-Lab
plugins: anyio-4.10.0
collected 1 item

demo9.py .                                                                                                                        [100%]

========================================================== 1 passed in 0.10s ===========================================================
PS C:\Users\HP\Desktop\AI-Lab>
```

Task - 8

Write a function LeapYearChecker and validate its implementation using 10 pytest test cases

```python


def LeapYearChecker(year):
    if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
        return True
    return False
def test_LeapYearChecker():
    assert LeapYearChecker(2020)==True
    assert LeapYearChecker(1900)==False
    assert LeapYearChecker(2000)==True
    assert LeapYearChecker(2021)==False
    print("All test cases for LeapYearChecker passed!")


```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\HP\Desktop\AI-Lab> python -m pytest demo9.py
=========== test session starts ===========
platform win32 -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\HP\Desktop\AI-Lab
plugins: anyio-4.10.0
collected 1 item

demo9.py .                                [100%]

============ 1 passed in 0.06s ============
PS C:\Users\HP\Desktop\AI-Lab>
```

Task - 9

Write a function SumOfDigits and validate its implementation using 7 pytest test cases.

```python
def SumOfDigits(num):
    return sum(int(digit) for digit in str(abs(num)))
def test_sum_of_digits():
    assert SumOfDigits(123)==6
    assert SumOfDigits(-456)==15
    assert SumOfDigits(0)==0
    assert SumOfDigits(9999)==36
    assert SumOfDigits(-12345)==15
    print("All test cases for SumOfDigits passed!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\HP\Desktop\AI-Lab> python -m pytest demo9.py
=========== test session starts ===========
platform win32 -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\HP\Desktop\AI-Lab
plugins: anyio-4.10.0
collected 1 item

demo9.py .                                [100%]

============ 1 passed in 0.10s ============
PS C:\Users\HP\Desktop\AI-Lab>
```

Task - 10

Write a function SortNumbers (implement bubble sort) and validate its implementation using 25 pytest test cases.

```python
def SortNumbers(numbers):
    n=len(numbers)
    for i in range(n):
        for j in range(0,n-i-1):
            if numbers[j] > numbers[j+1]:
                numbers[j],numbers[j+1]=numbers[j+1],numbers[j]
    return numbers
def test_SortNumbers():
    assert SortNumbers([5,2,9,1,5,6])==[1,2,5,5,6,9]
    assert SortNumbers([])==[]
    assert SortNumbers([3])==[3]
    assert SortNumbers([3,2])==[2,3]
    assert SortNumbers([10,9,8,7])==[7,8,9,10]
    assert SortNumbers([-1,-3,0,2])==[-3,-1,0,2]
    assert SortNumbers([1,2,3,4,5])==[1,2,3,4,5]
    assert SortNumbers([5,4,3,2,1])==[1,2,3,4,5]
    assert SortNumbers([1,1,1,1])==[1,1,1,1]
    assert SortNumbers([3,1,2,3,1])==[1,1,2,3,3]
    assert SortNumbers([0,0,0,0])==[0,0,0,0]
    assert SortNumbers([1,2,3,4,5,6,7,8,9,10])==[1,2,3,4,5,6,7,8,9,10]
    print("All test cases for SortNumbers passed!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
========== test session starts ==========
platform win32 -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\HP\Desktop\AI-Lab
plugins: anyio-4.10.0
collected 1 item

demo9.py .                                [100%]

========== 1 passed in 0.07s ==========
```

Task - 11

Write a function ReverseString and validate its implementation using 5 unittest test cases

```python

def ReverseString(s):
    return s[::-1]
import unittest
class TestReverseString(unittest.TestCase):
    def test_reverse_string(self):
        self.assertEqual(ReverseString("Hello"), "olleH")
        self.assertEqual(ReverseString("Python"), "nohtyP")
        self.assertEqual(ReverseString(""), "")
        self.assertEqual(ReverseString("A"), "A")
        self.assertEqual(ReverseString("12345"), "54321")
if __name__ == '__main__':
    unittest.main()

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\HP\Desktop\AI-Lab> & C:/Users/HP/AppData/Local/Programs/Python/Python313/python.exe c:/Users/H
.
----------------------------------------------------------------------
Ran 1 test in 0.001s

OK
PS C:\Users\HP\Desktop\AI-Lab>
```

Task - 12

Write a function AnagramChecker and validate its implementation using 10 unittest test cases.

```python

def AnagramChecker(str1,str2):
    return sorted(str1.replace(" ","").lower()) == sorted(str2.replace(" ","").lower())
import unittest
class TestAnagramChecker(unittest.TestCase):
    def test_anagram_checker(self):
        self.assertTrue(AnagramChecker("listen","silent"))
        self.assertTrue(AnagramChecker("Triangle","Integral"))
        self.assertFalse(AnagramChecker("Hello","World"))
        self.assertTrue(AnagramChecker("Dormitory","Dirty Room"))
        self.assertFalse(AnagramChecker("Python","Java"))
        self.assertTrue(AnagramChecker("state","taste"))
        self.assertFalse(AnagramChecker("Conversation","Voices Rant ON"))
        self.assertTrue(AnagramChecker("School master","The classroom"))
if __name__ == '__main__':
    unittest.main()

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
    ^^^
```

Task - 13

Write a function ArmstrongChecker and validate its implementation using 8 unittest test cases.

```python
def ArmStrongNumberChecker(num):
        num_str=str(num)
        num_digits=len(num_str)
        armstrong_sum=sum(int(digit)**num_digits for digit in num_str)
        return armstrong_sum == num
import unittest
class TestArmStrongNumberChecker(unittest.TestCase):
    def test_armstrong_number_checker(self):
        self.assertTrue(ArmStrongNumberChecker(153))
        self.assertTrue(ArmStrongNumberChecker(370))
        self.assertTrue(ArmStrongNumberChecker(371))
        self.assertTrue(ArmStrongNumberChecker(407))
        self.assertFalse(ArmStrongNumberChecker(123))
        self.assertFalse(ArmStrongNumberChecker(0))
        self.assertFalse(ArmStrongNumberChecker(-153))
if __name__ == '__main__':
    unittest.main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
    ^^^
```